ASSIGNMENT 5

Database and Table Creation

Event Ticketing and Venue Seating System

Name	Aishani Dutta
Registration Number	2511021240020
Course / Section	BCA SEM 3 2025
Date	23 October 2026

The database name is EventTicketingDB. The seven tables follow the event ticketing design. MySQL 8.0.16 or later is required for the CHECK constraints used here.

Create the database

```
CREATE DATABASE EventTicketingDB CHARACTER SET utf8mb4;
USE EventTicketingDB;
```

Run the creation script once in a fresh database. Create parent tables before tables that refer to them. InnoDB is used because it supports transactions and Foreign Key constraints.

1. Users

```
CREATE TABLE Users (
    user_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    phone VARCHAR(20) NOT NULL UNIQUE,
    email VARCHAR(150) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;
```

Users holds customer details. Phone and email cannot be shared by two accounts in this design. created_at is filled automatically when it is omitted.

2. Venues

```
CREATE TABLE Venues (
    venue_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    address VARCHAR(200) NOT NULL,
    city VARCHAR(60) NOT NULL,
    total_capacity INT NOT NULL,
    CONSTRAINT positive_capacity CHECK (total_capacity > 0)
) ENGINE=InnoDB;
```

Venues stores each location. total_capacity must be greater than zero.

3. Seats

```
CREATE TABLE Seats (
    seat_id INT PRIMARY KEY AUTO_INCREMENT,
    venue_id INT NOT NULL,
    section VARCHAR(40) NOT NULL,
    row_label VARCHAR(10) NOT NULL,
    seat_number VARCHAR(10) NOT NULL,
    seat_type VARCHAR(15) NOT NULL DEFAULT 'Regular',
    CONSTRAINT seat_location UNIQUE
        (venue_id, section, row_label, seat_number),
    CONSTRAINT valid_seat_type CHECK
        (seat_type IN ('Regular', 'Premium', 'VIP')),
    FOREIGN KEY (venue_id) REFERENCES Venues(venue_id)
        ON DELETE RESTRICT
) ENGINE=InnoDB;
```

Seats stores the seating plan. The same row and seat number can be used at another venue, but cannot be repeated within the same venue and section.

4. Events

```
CREATE TABLE Events (
    event_id INT PRIMARY KEY AUTO_INCREMENT,
    venue_id INT NOT NULL,
    title VARCHAR(150) NOT NULL,
    category VARCHAR(40) NOT NULL,
    event_date DATE NOT NULL,
    start_time TIME NOT NULL,
    description VARCHAR(255),
    FOREIGN KEY (venue_id) REFERENCES Venues(venue_id)
        ON DELETE RESTRICT
) ENGINE=InnoDB;
```

Events contains the show details. One venue can host several events on different dates.

5. Bookings

```
CREATE TABLE Bookings (
    booking_id INT PRIMARY KEY AUTO_INCREMENT,
    user_id INT NOT NULL,
    event_id INT NOT NULL,
    booking_time DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    total_amount DECIMAL(10,2) NOT NULL DEFAULT 0.00,
    booking_status VARCHAR(15) NOT NULL DEFAULT 'Pending',
    UNIQUE KEY booking_event (booking_id, event_id),
    CONSTRAINT nonnegative_total CHECK (total_amount >= 0),
    CONSTRAINT valid_booking_status CHECK
        (booking_status IN ('Pending', 'Confirmed', 'Cancelled')),
    FOREIGN KEY (user_id) REFERENCES Users(user_id)
        ON DELETE RESTRICT,
    FOREIGN KEY (event_id) REFERENCES Events(event_id)
        ON DELETE RESTRICT
) ENGINE=InnoDB;
```

Bookings records the customer, event and booking status. One booking can contain several seats. DECIMAL stores monetary amounts without floating-point rounding.

6. Booking_Seats

```
CREATE TABLE Booking_Seats (
    booking_seat_id INT PRIMARY KEY AUTO_INCREMENT,
    booking_id INT NOT NULL,
    seat_id INT NOT NULL,
    event_id INT NOT NULL,
    price DECIMAL(10,2) NOT NULL,
    ticket_code VARCHAR(40) NOT NULL UNIQUE,
    CONSTRAINT one_seat_per_event UNIQUE (event_id, seat_id),
    CONSTRAINT nonnegative_price CHECK (price >= 0),
    FOREIGN KEY (booking_id, event_id)
        REFERENCES Bookings(booking_id, event_id)
        ON DELETE RESTRICT,
    FOREIGN KEY (seat_id) REFERENCES Seats(seat_id)
        ON DELETE RESTRICT,
    FOREIGN KEY (event_id) REFERENCES Events(event_id)
        ON DELETE RESTRICT
) ENGINE=InnoDB;
```

Booking_Seats contains one row per selected seat. The pair (event_id, seat_id) is unique. The combined Foreign Key ensures that booking_id and event_id belong together.

7. Payments

```
CREATE TABLE Payments (
    payment_id INT PRIMARY KEY AUTO_INCREMENT,
    booking_id INT NOT NULL,
    amount DECIMAL(10,2) NOT NULL,
    payment_method VARCHAR(20) NOT NULL,
    payment_status VARCHAR(15) NOT NULL DEFAULT 'Pending',
    paid_at DATETIME,
    CONSTRAINT positive_payment CHECK (amount > 0),
    CONSTRAINT valid_payment_method CHECK
        (payment_method IN ('UPI', 'Card', 'Net Banking')),
    CONSTRAINT valid_payment_status CHECK
        (payment_status IN ('Pending', 'Successful',
                            'Failed', 'Refunded')),
    CONSTRAINT payment_time CHECK
        (payment_status NOT IN ('Successful', 'Refunded')
         OR paid_at IS NOT NULL),
    FOREIGN KEY (booking_id) REFERENCES Bookings(booking_id)
        ON DELETE RESTRICT
) ENGINE=InnoDB;
```

Payments stores payment attempts. A booking may have a failed attempt followed by a successful attempt, so booking_id is not unique in Payments.

Table relationships

Parent	Related table	Relationship
Users	Bookings	One-to-many
Venues	Seats	One-to-many
Venues	Events	One-to-many
Events	Bookings	One-to-many
Bookings	Booking_Seats	One-to-many
Seats	Booking_Seats	One-to-many across events
Events	Booking_Seats	One-to-many
Bookings	Payments	One-to-many

Tables created

```
SHOW TABLES;
```

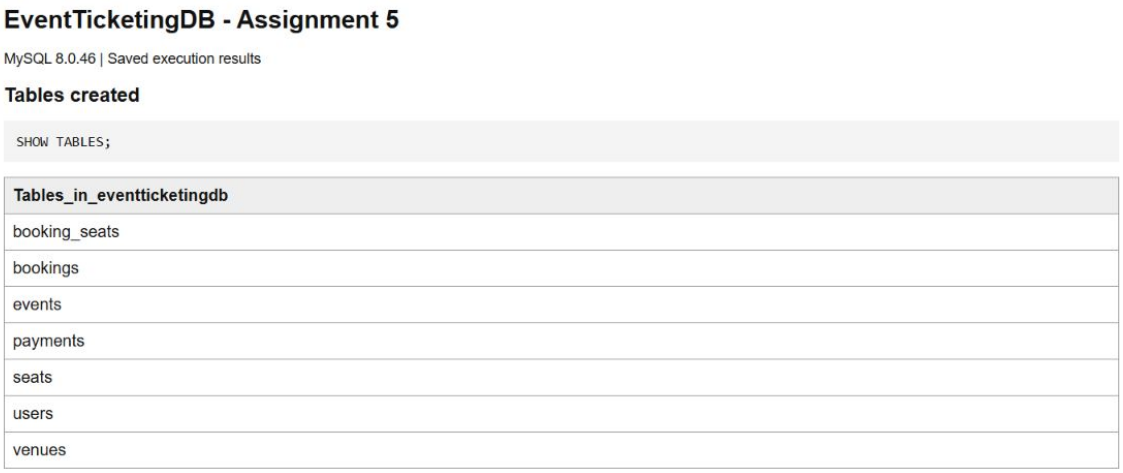
Output

Tables_in_eventticketingdb
booking_seats
bookings
events
payments
seats
users
venues

All seven tables are present.

The cross-table venue checks are added in Assignment 6. All tables were created successfully in MySQL 8.0.46.

Query result screenshot



EventTicketingDB - Assignment 5

MySQL 8.0.46 | Saved execution results

Tables created

```
SHOW TABLES;
```

Tables_in_eventticketingdb
booking_seats
bookings
events
payments
seats
users
venues

Saved MySQL results displayed in the browser.